

Lista Linear

Alocação Sequencial

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

Definição – Lista Linear

- Uma lista linear ou tabela é um conjunto de $n \geq 0$ nós, $L[1], L[2], \dots, L[n]$, tais que suas propriedades estruturais decorrem unicamente da posição relativa dos nós dentro da sequência linear. Tem-se que:
 - Se $n > 0$, $L[1]$ é o primeiro nó;
 - Para $1 < k \leq n$, o nó $L[k]$ é precedido do nó $L[k-1]$;
 - Todos os nós são do mesmo tipo.
- Operações mais frequentes:
 - Busca – localizar o i -ésimo item;
 - Inserir após o i -ésimo nó;
 - Retirar o i -ésimo item.

Definição – Lista Linear

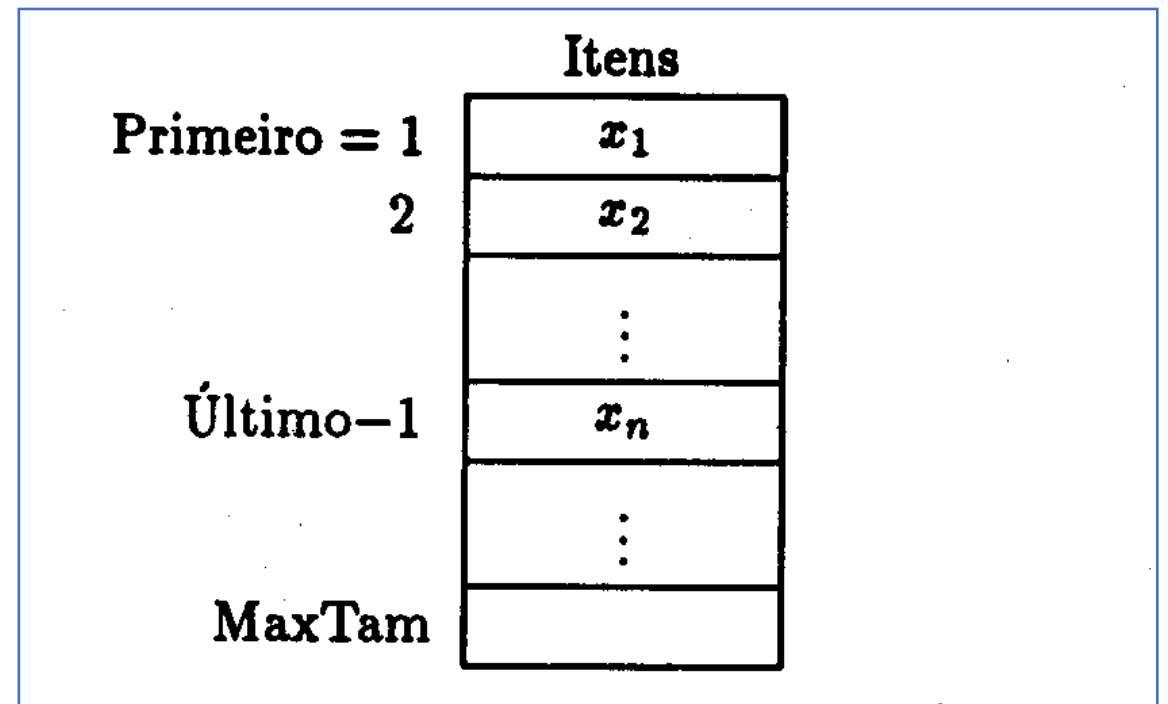
- Outras operações possíveis: criar uma lista vazia, alteração de um elemento, combinação de listas, ordenação dos nós, determinar o primeiro ou último nó, determinar a cardinalidade, etc.
- Uma lista linear armazenada, somada ao conjunto de operações implementadas, forma um tipo abstrato de dados Lista.

Armazenamento

- Classificado de acordo com a posição relativa de dois nós consecutivos na memória.
- Alocação sequencial – nós consecutivos ocupam posições sequenciais na memória.
- Alocação encadeada – nós consecutivos não ocupam posições sequenciais na memória, sendo necessário alguma forma de indicar a sequência dos nós.

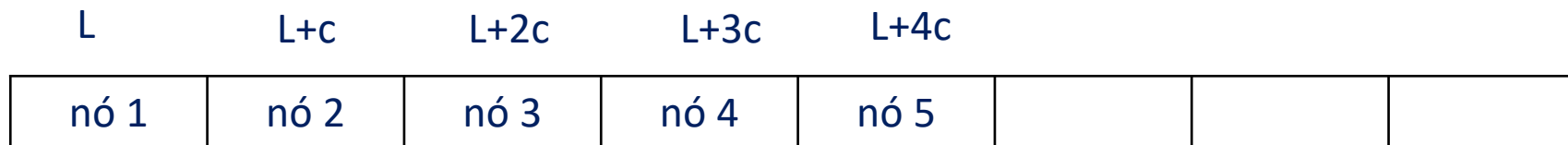
Alocação sequencial

- Nós consecutivos são alocados em posições contíguas. Utiliza arranjos ou vetores.



Alocação sequencial

- Endereço real de memória do $(j+1)$ -ésimo nó da lista se encontra **c** unidades adiante do endereço correspondente ao j -ésimo nó, sendo a constante **c** o número de palavras de memória ocupadas por um nó.



Alocação sequencial

- Cada nó pode conter vários campos, sendo que todos os nós possuem a mesma estrutura.

chave	nome	endereço
-------	------	----------

Algoritmo

```
lista array[10] int  
n, m int  
n = 0  
m = 10
```

Em Python

```
lista = [0,0,0,0,0,0,0,0,0,0]  
n = 0  
m = 10
```

Busca de um elemento em uma lista

- Busca de um nó na lista **L** conhecendo o seu valor de chave, o qual é armazenado na variável **x**.

```
function busca(x)
    i = 1; busca = 0
    while i ≤ n do
        if L[i].chave == x then
            busca = i
            i = n + 1
        else i = i + 1
    return i
```

Em Python:

```
def buscar(chave, n):
    ind = 0
    while ((ind ≤ n) and
           (lista[ind] != chave)):
        ind = ind + 1
    return ind
```


Busca de um elemento em uma lista

- Busca de um nó na lista **L** conhecendo o seu valor de chave, o qual é armazenado na variável **x**.

```
function busca(x)
    i = 1; busca = 0
    while i ≤ n do
        if L[i].chave == x then
            busca = i
            i = n + 1
        else i = i + 1
    return i
```

Complexidade é $O(n)$

Busca de um elemento em uma lista

- Algoritmo alternativo.

```
function busca(x)
    i = 1; busca = 0
    while i ≤ n and L[i].chave != x do
        i = i + 1
    if i ≤ n then
        busca = i
    return busca
```

Busca de um elemento em uma lista – algoritmo alternativo

- Algoritmo com execução mais rápida:

```
function busca(x)
    L[n+1].chave = x; i = 1
    while L[i].chave ≠ x do
        i = i + 1
    if i ≠ n+1 then
        busca = i
    else busca = 0
    return busca
```

Em Python:

```
def buscaralt(chave, n):
    lista[n] = chave
    ind = 0
    while ((ind < n)):
        ind = ind + 1
    return ind
```

Busca de um elemento em uma lista – algoritmo alternativo

- Algoritmo com execução mais rápida:

```
function busca(x)
    L[n+1].chave = x; i = 1
    while L[i].chave ≠ x do
        i = i + 1
    if i ≠ n+1 then
        busca = i
    else busca = 0
    return busca
```

Complexidade é $O(n)$

Mais rápido porque
executa um teste a
menos em cada
iteração!

Busca em uma lista não ordenada em C

```
int busca(int x, int *lista, int n)
{
    int i;
    i = 0;
    while ((i<n) and (lista[i]!=x))
        i=i+1;
    if (i == n)
        i = -1;
    return i;
}
```

```
int lista[30]
```

Complexidade média da busca linear

- Caso a chave sempre se encontre na lista, e assumindo probabilidades iguais de estar em qualquer posição.
 - Com n elementos, a probabilidade de qualquer elemento ser igual à chave é: $q = n/2$.
- Caso a chave não se encontre na lista, os n elementos serão percorridos e a probabilidade será de n .
- Caso existam probabilidades iguais de a chave ser encontrada na lista ou não, temos que a complexidade média será de $3n/4$.

Inserção de um nó

- A lista não está ordenada. A memória alocada tem um máximo de M posições.

```
if n < M then
    if busca(x) == 0 then
        L[n+1] = novo-valor
        n = n + 1
    else "elemento já existe"
else overflow
```

```
def incluir(chave,n, m):
    ind = 0
    if n<m:
        ind = buscar(chave, n)
        if (ind==n):
            lista[n]=chave
            n = n + 1
        else:
            print("Numero existe na lista.")
        else:
            print("Overflow")
    return n
```

Inserção de um nó

- A lista não está ordenada. A memória alocada tem um máximo de M posições.

```
if  $n < M$  then
    if busca(x) == 0 then
         $L[n+1] = \text{novo-valor}$ 
         $n = n + 1$ 
    else "elemento já existe"
else overflow
```

Este procedimento é simples, porém depende do algoritmo de busca. A complexidade é $O(n)$.

Remoção de um nó

- A lista não está ordenada. A memória alocada tem um máximo de M posições.

```
if  $n \neq 0$  then
    índice = busca(x)
    if índice  $\neq 0$  then
        valor-recuperado = L[índice]
        for i = índice, n-1 do
            L[i] = L[i+1]
        n = n - 1
    else "elemento não se encontra na tabela"
else underflow
```

Remoção de um nó

- A lista não está ordenada. A memória alocada tem um máximo de M posições.

```
if n ≠ 0 then
    índice = busca(x)
    if índice ≠ 0 then
        valor-recuperado = L[índice]
        for i = índice, n-1 do
            L[i] = L[i+1]
        n = n - 1
    else "elemento não se encontra na tabela"
else underflow
```

A complexidade é $O(n)$.
No pior caso percorre a lista 2 vezes, sendo $2n$, sendo que constantes multiplicativas são descartadas.

- Remoção em Python

```
def excluir(chave, n):  
    ind = 0  
    i = 0  
    if (n>0):  
        ind = buscar(chave,n)  
        if ((ind<n)):  
            for i in range (ind,n):  
                lista[i] = lista[i+1]  
            n = n - 1  
        else:  
            print("Numero nao existe na lista.")  
    else:  
        print("Underflow")  
    return n
```

- Remoção em Python – versão alternativa

```
def excluir(chave, n):
```

```
    ind = 0
```

```
    i = 0
```

```
    if (n>0):
```

```
        ind = buscar(chave,n)
```

```
        if ((ind<n)):
```

```
            lista[ind] = lista[n-1]
```

```
            n = n - 1
```

```
        else:
```

```
            print("Numero nao existe na lista.")
```

```
    else:
```

```
        print("Underflow")
```

```
    return n
```

Busca em lista ordenada

- Assumindo que a lista está ordenada em ordem crescente da chave.

```
function busca-ord(x)
    L[n+1].chave = x; i = 1
    while L[i].chave < x do
        i = i + 1
    if i == n+1 or L[i].chave ≠ x then
        busca = 0
    else busca = i
    return busca
```

Em Python:

```
def buscar(chave, n):
    ind = 0
    while ((ind < n) and (lista[ind] < chave)):
        ind = ind + 1
    return ind
```

Busca em lista ordenada

- Assumindo que a lista está ordenada em ordem crescente da chave.

```
function busca-ord(x)
    L[n+1].chave = x; i = 1
    while L[i].chave < x do
        i = i + 1
    if i == n+1 or L[i].chave ≠ x then
        busca = 0
    else busca = i
    return busca
```

Complexidade é $O(n)$

A complexidade média é de aproximadamente $n/2$, assumindo probabilidades iguais de a chave se encontrar ou não na lista, e caso se encontre, existe igual probabilidade de estar em qualquer posição, e caso não se encontre, também igual probabilidade de o valor estar em qualquer intervalo de valores da lista.

Lista Linear com alocação sequencial - ordenada

- Inserção em lista linear ordenada.

```
function busca-ord(x)
    i = 1
    while (i <= n) and (L[i].chave < x) do
        i = i + 1
    return i
```

```
if n < M then
    indice = busca-ord(x)
    if indice > n or L[indice] > x then
        for i = n to indice (-1) do
            L[i+1] = L[i]
        L[indice] = x
        n = n + 1
    else "elemento se encontra na tabela"
else overflow
```

Lista Linear com alocação sequencial - ordenada

- Exclusão em lista linear ordenada.

```
function busca-ord(x)
    i = 1
    while (i <= n) and (L[i].chave < x) do
        i = i + 1
    return i
```

```
if n > 0 then
    indice = busca-ord(x)
    if L[indice] == x then
        valor = L[indice]
        for i = índice to n do
            L[i] = L[i + 1]
        n = n - 1
    else "elemento não encontrado"
else underflow
```


- Inclusão em lista linear ordenada em Python

```
def incluir(chave,n, m):  
    ind = 0  
    if n<m:  
        ind = buscar(chave, n)  
        if (ind==n):  
            lista[n]=chave  
            n = n + 1  
        else:  
            if (lista[ind] != chave):  
                i = n-1  
                while i>=ind:  
                    lista[i+1] = lista[i]  
                    i=i-1  
                lista[ind] = chave  
                n = n + 1  
            else:  
                print("Numero existe na lista.")  
    else:  
        print("Overflow")  
    return n
```

- Remoção em lista linear ordenada em Python

```
def excluir(chave, n):  
    ind = 0  
    i = 0  
    if (n>0):  
        ind = buscar(chave,n)  
        if ((ind<n) and (lista[ind]==chave)):  
            for i in range (ind,n):  
                lista[i] = lista[i+1]  
            n = n - 1  
        else:  
            print("Numero nao existe na lista.")  
    else:  
        print("Underflow")  
    return n
```

Busca em lista sequencial ordenada – em C

```
int busca_ord(int x, int * listaord)
{
    int i;
    i = 0;
    while ((i<20) and (listaord[i]< x))
        i=i+1;
    if (listaord[i]=x)
        return i;
    else
        return -1;
}
```

Busca binária

- A lista ordenada é percorrida de forma similar à que se folheia um dicionário.

```
function busca-bin(x)
    inf = 1; sup = n; busca = 0
    while inf ≤ sup do
        meio = ⌊(inf + sup)/2⌋
        if L[meio].chave == x then
            busca = meio
            inf = sup + 1
        else se L[meio].chave < x então
            inf = meio + 1
        else sup = meio - 1
    return busca
```

Busca binária

- A lista ordenada é percorrida de forma similar à que se folheia um dicionário.

```
function busca-bin(x)
    inf = 1; sup = n; busca = 0
    while inf ≤ sup do
        meio = ⌊(inf + sup)/2⌋
        if L[meio].chave == x then
            busca = meio
            inf = sup + 1
        else se L[meio].chave < x então
            inf = meio + 1
        else sup = meio - 1
    return busca
```

A cada iteração metade da lista é descartada, dividindo a lista ao meio, até encontrar a chave, ou até que se fique com **1** elemento. O número de iteração será igual a **$1 + \lfloor \log_2 n \rfloor$** .
A complexidade será **$O(\log n)$**

Busca Binária em lista sequencial ordenada – em Python

```
def buscabinaria(chave, n):  
    inf = 0;  
    sup = n-1;  
    meio = 0;  
    while (inf <= sup):  
        meio = (inf + sup) // 2;  
        if (lista[meio] == chave):  
            inf = sup + 1  
        else:  
            if (chave < lista[meio]):  
                sup = meio - 1  
            else:  
                inf = meio + 1;  
            if (lista[meio] == chave):  
                return meio  
        else:  
            return -1;
```

Busca Binária em lista sequencial ordenada – em C

```
int busca_bin(int x, int * listaord)
{
    int inf, sup, meio;
    inf = 0;
    sup = 19;
    while (inf <= sup)
    {
        meio = (inf + sup) / 2;
        if (listaord[meio] == x)
            inf = sup + 1;
        else
            if (x < listaord[meio])
                sup = meio - 1;
            else
                inf = meio + 1;
    }
    if (listaord[meio] == x)
        return meio;
    else
        return -1;
}
```